

Edge AI Computing for Grape Leaf Disease Detection



UNIVERSITÀ DI PISA



Thesis Progress...

February 8, 2026

Updates:

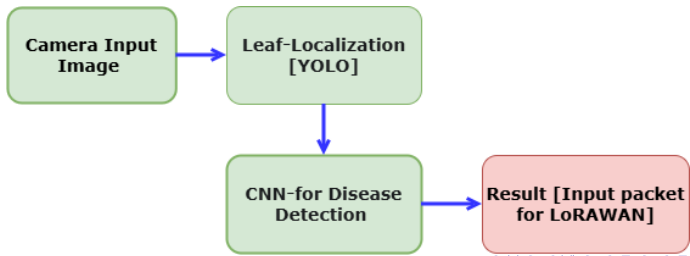
- 1 Previous Recap
- 2 System Architecture Overview
- 3 Leaf Localization ESPDet-Pico Model (YOLO)
- 4 Disease Classification Model (MobileNetV2)
- 5 Integration and Deployment of lightweight models in ESP32-S3
 - Model Integration pipeline testing (pre-deployment)
 - ESP32-S3 Hardware Deployment and inference
- 6 Result and conclusion
- 7 Next Steps: Integrating to the LoRaWAN
- 8 References

Proposed System Architecture

- *Four-Stage Pipeline:*

- Stage 1: Input image captured using ESP32-S3 camera
- Stage 2: Localization: A lightweight model to detect leaf region (YOLO)
- Stage 3: Classification: Disease detection model (MobileNetV2)
- Stage 4: Sending only the classification result via LoRaWAN

Current Implementation Status: - Stage 1–3 **Completed**
- Stage 4 **In-progress**

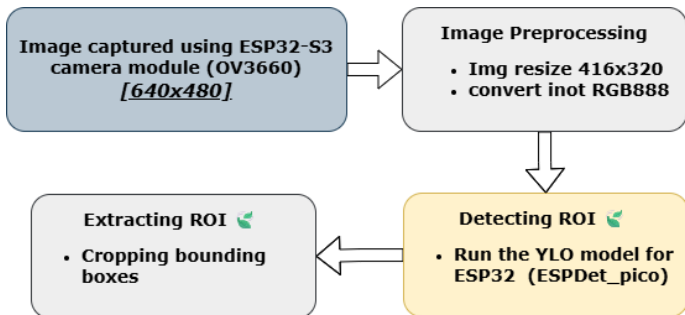


Previous Recap

- **MobileNetV2** architecture was successfully fine-tuned and validated on the standard Kaggle dataset. It achieved high accuracy across the four target leaf disease classes (Black Rot, Esca, Healthy, Leaf Blight).
- **ESP32-S3 WROOM-1** dev kit was configured for camera capture and runtime deployment.
- A realistic vine-tree dataset was collected, preprocessed and annotated via Roboflow for leaf ROI detection.

Stage 2: Leaf Localization ESPDet-Pico Model (YOLO)

The main objective of this stage is to **detect and localize grape leaf**, which is the **Region of Interest (ROI)** for disease detection. By cropping the ROI bounding boxes the disease detection model receives a clean and relevant input.



Dataset Preparation & Annotation

The "Grape/Grapes" dataset from Kaggle (authored by Nicolaas Regnier) contains 531 different image instances and augmented into 1231. Unlike standard datasets, which are single leaves on uniform backgrounds, this collection captures the **unstructured complexity** of a real vineyard.



Realistic Conditions:

- Include different backgrounds
- Multiple overlapping leaves per image
- Resized to mimics the ESP32-S3 camera input

Source: Kaggle (N. Regnier)

Annotation Tool and Workflow

The dataset was annotated using Roboflow with Meta Segment Anything (SAM) assisted labeling, exported in YOLOv11 in **YOLOv11** bounding box format.



(a) Vine tree image



(b) Annotated leaves on the vine tree image in Roboflow using SAM model

ESPDet-Pico Architecture

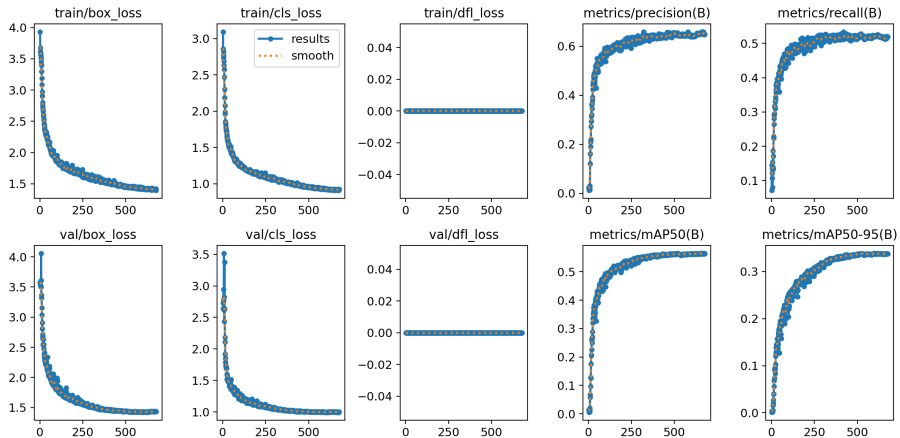
The localization model is designed specifically for resource constrained ESP32 devices. Therefore, the architecture selection, training strategy, and deployment pipeline prioritize size, latency, and compatibility with Espressifs ESP-DL runtime.

Therefore for the device we have (ESP32-S3) the ESPDet-Pico Architecture is optimized for embedded inference using compact convolutional blocks and an efficient detection head compatible with ESP-DL.

ESPDet-Pico model characteristics:

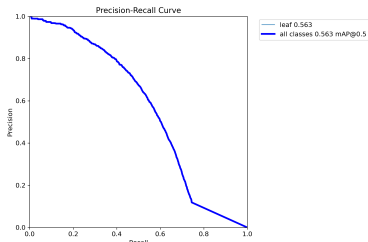
- it's a lightweight YOLO11-style detector optimized for ESP-DL
- it has 361,167 (**$\approx 0.36\text{M}$**) number of parameters
- input 416×320 RGB image
- single-class object detection
- outputs bounding boxes for each ROIs (leaves) in the input image

ESPDet-Pico Training

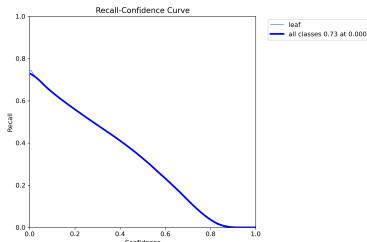


- Training/validation losses decrease smoothly → stable convergence.
- Final Precision ≈ 0.65 / Recall ≈ 0.52 .
- Final detection performance: mAP@0.5 ≈ 0.56 , mAP@0.5:0.95 ≈ 0.34 .

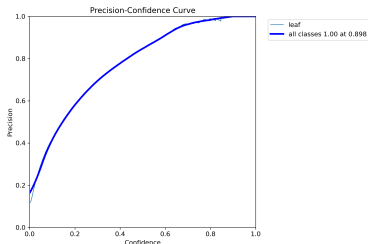
ESPDet-Pico Evaluation Curves



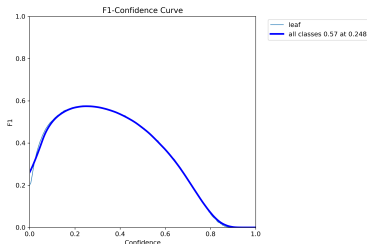
(a) PR curve ($\text{mAP}@0.5 \approx 0.563$).



(b) Recall vs conf (max ≈ 0.73 at conf=0.0).



(c) Precision vs conf ($P \approx 1.0$ at conf ≈ 0.898).



(d) F1 vs conf (best F1 ≈ 0.57 at conf ≈ 0.248).

Note: The F1 curve is used to select a practical confidence threshold for deployment.

Model Export to ONNX Format and Quantization to INT8 (ESP-PPQ)

- **ONNX Export:** After training the ESPDet-Pico leaf localization model, the PyTorch weights were exported into the ONNX (Open Neural Network Exchange) format to obtain a standardized intermediate representation. The export used a fixed input shape of [1, 3, 320, 416] (NCHW) and opset 13 to ensure compatibility with the ESP-DL pipeline.
- **INT8 Quantization (ESP-PPQ):** The ONNX model (1.8 MB, FP32) was quantized to INT8 using ESP-PPQ with 32 calibration batches and equalization enabled. This achieved 73% compression (**1.8MB → 478KB**), reducing memory and computational cost.
- **ESP-DL Deployment:** The quantized model was converted to **.espdl** format (**478 KB, INT8**), producing a deployment-ready artifact for ESP32-S3. This enables on-device inference with **≈330ms latency** per frame, supporting real-time leaf localization.

Training Pipeline

1. Dataset Preparation (Roboflow)

1230 images (1,077 train + 103 val + 50 test) — YOLOv11 format
— Single class: 'leaf'



2. Model Training (ESPDet-Pico)

677 epochs — AdamW optimizer — Precision: 64.8%, mAP50: 56.4%



3. ONNX Export

PyTorch → ONNX (FP32, 1.8 MB) — ESP-specific modifications



4. INT8 Quantization (ESP-PPQ)

32 calibration batches — Equalization enabled



5. ESP-DL Deployment Model (.espdll)

478 KB — INT8 — ESP32-S3 ready — Inference: ≈ 330 ms/frame

Disease Classification Model (MobileNetV2)

Previous Work

- MobileNetV2 was trained for grape disease detection (4 classes)
- Input size: $224 \times 224 \times 3$ RGB
- Model size: 8.75 MB (.pth)
- High accuracy achieved

Challenge

Model was too large for ESP32-S3 (8 MB PSRAM).

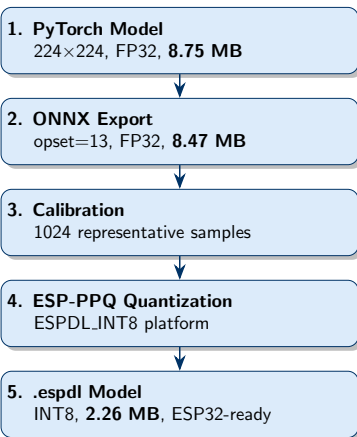
- Cannot fit the PyTorch model and activations
- 224×224 input requires significant memory

Actions Taken

- Model is quantized and converted into a compatible format
- Input size is reduced: $224 \times 224 \rightarrow 128 \times 128$

Model Quantization

To deploy the MobileNetV2 disease classification model on the ESP32-S3, I exported the trained network into ONNX format and then quantized to **INT8** using **ESP-PPQ**. I used similar quantization configuration for both models, ensuring ESP-DL compatibility.



Statistics:

- 277 variables quantized
- 100 operations quantized
- Per-layer optimization

Input Size Optimization

Retrained model with 128×128 input (from 224×224)

- 67% reduction in input pixels ($150,528 \rightarrow 49,152$)
- Test accuracy stayed preserved after resolution reduction
- Optimize memory usage and inference latency

128×128 Input Achieves Same Accuracy!

Metric	224×224	128×128
Input tensor elements ($H \times W \times 3$)	150,528	49,152
Input reduction	–	-67%
Test Accuracy	100%	100%
Memory Usage	Higher	67% lower
Inference Latency	Baseline	Faster
Preprocessing	Baseline	$\approx 3 \times$ faster

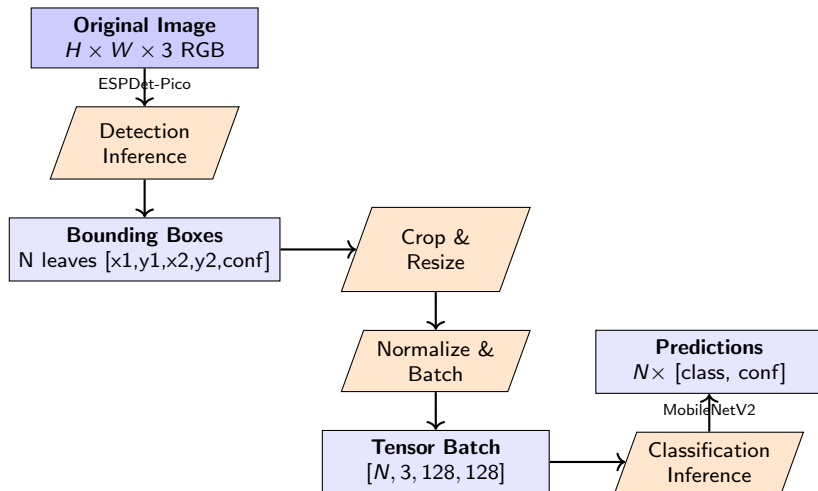
Model Integration pipeline testing

- I have created a Jupyter Notebook for testing the integrated pipeline (leaf localization + disease classification) using static input images.
- Inference is executed using the original **FP32 PyTorch models** (before quantization) on GPU resources to validate end-to-end functionality prior to deployment on the **ESP32-S3**.

Pipeline Steps:

- 1 **ROI Detection (ESPDet-Pico):** Input image $\rightarrow$ resize $[N, 3, 320, 416] \rightarrow$ detect leaf bounding boxes (ROIs).
- 2 **Intermediate Preprocessing:** Crop detected leaf ROIs $\rightarrow$ resize to $128 \times 128 \rightarrow$ normalize and batch into tensor $[N, 3, 128, 128]$.
- 3 **Disease Detection (MobileNetV2):** Run classification inference on each ROI $\rightarrow$ output class label and confidence score.

Pipeline Data Flow



Input Image (Pipeline Testing)

Input image



Figure: Static vine-tree image used for pre-deployment pipeline testing (the real camera camera integration is implemented after this).

Leaf ROI Detection (ESPDet-Pico)



Figure: Detected leaf ROIs using ESPDet-Pico with detection confidence threshold ≥ 0.35 . Each bounding box represents a candidate leaf region to be cropped and forwarded to the disease classification stage.

Disease Diagnosis per Detected Leaf (MobileNetV2)

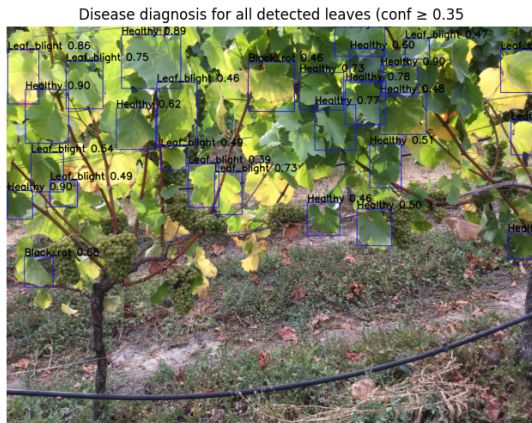


Figure: Disease prediction and confidence score for each detected leaf ROI.

Leaf crops are resized to 128×128 and classified using MobileNetV2, producing per-leaf disease labels and confidences.

Deployment Hardware setup

Components of the ESP32-S3 WROOM-1 dev kit

Processing Unit:

- Dual-core Xtensa LX7 @ 240 MHz
- AI acceleration support
- Vector instructions for ML

Memory Configuration:

- 512 KB SRAM (internal)
- 8 MB PSRAM (external)
- 16 MB Flash storage

Camera Interface:

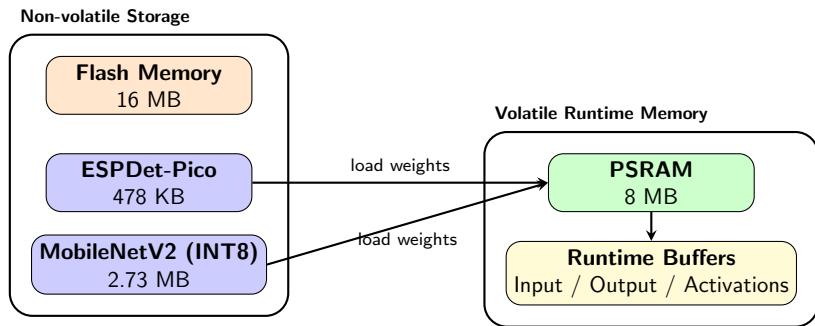
- OV3660 sensor
- 640×480 resolution
- JPEG (Q=12)

Power Characteristics:

- Active: ~ 240 mA ~ 3.3 V
- Battery-powered capable

Models Deployment Architecture

*After training, INT8 quantization, conversion to .espdl, and the integrated inference (pre-deployment testing) of the two models, in the target **ESP32-S3** resource constrained device, deployed as follow:*

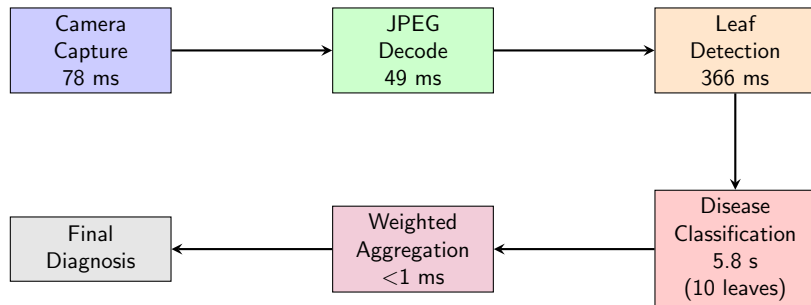


- Models packed into a single binary (`grape_leaf_detect.espdl`) package and stored in flash-RAM
- PSRAM used for runtime tensors, buffers, and activations

Memory allocation strategy for complete inference pipeline

Component	Size	Location	Usage
Bootloader	23 KB	Flash	Boot sequence
Partition Table	3 KB	Flash	Memory map
Application Code	4.4 MB	Flash	Firmware
ESPDet-Pico	478 KB	Flash (packed)	Leaf detection
MobileNetV2	2.73 MB	Flash (packed)	Disease classification
DMA Buffers (16 nodes)	16 KB	SRAM	Hardware staging
JPEG Frame Buffers (2×)	120 KB	PSRAM	Double buffering
RGB888 Decoded	921 KB	PSRAM	$640 \times 480 \times 3$
Detection Input	492 KB	PSRAM	$416 \times 320 \times 3$
Classification Input	49 KB	PSRAM	$128 \times 128 \times 3$
Model Activations	2.6 MB	PSRAM	Runtime tensors
Output Tensors	~16 KB	PSRAM	Logits
Total Used	4.1 MB	PSRAM	Active buffers
Free PSRAM	3.9 MB	PSRAM	Available

Inference Pipeline Performance



Total Pipeline Time: ~6.3 seconds per frame

- Image acquisition: 127 ms (2%)
- Leaf detection: 366 ms (6%)
- Disease classification: 5.8 s (92%)
- 535 ms per leaf $\times$ 10 leaves

Hardware Inference Testing

```
grape_leaf_DD: Captured: 640x480, 12199 bytes (0 ms)␣[0m
grape_leaf_DD: Decoded: 640x480 RGB888 (49 ms)␣[0m
grape_leaf_DD: Detected 5 objects (353 ms)␣[0m
grape_leaf_DD: Sorted by confidence (highest first)␣[0m
grape_leaf_DD: Running disease classification on top 3 detections:␣[0m
grape_leaf_DD:   [0] Bbox: [234,23,260,47] (624 px²), Leaf_Confidence: 0.438␣[0m
grape_leaf_DD:     Disease Probability:␣[0m
grape_leaf_DD:       • Healthy: 48.78%␣[0m
grape_leaf_DD:       • Leaf_blight: 30.53%␣[0m
grape_leaf_DD:       • Black_rot: 11.23%␣[0m
grape_leaf_DD:   [1] Bbox: [0,43,34,115] (2448 px²), Leaf_Confidence: 0.321␣[0m
grape_leaf_DD:     Disease Probability:␣[0m
grape_leaf_DD:       • Healthy: 62.28%␣[0m
grape_leaf_DD:       • Leaf_blight: 21.86%␣[0m
grape_leaf_DD:   [2] Bbox: [311,13,340,40] (783 px²), Leaf_Confidence: 0.269␣[0m
grape_leaf_DD:     Disease Probability:␣[0m
grape_leaf_DD:       • Healthy: 42.40%␣[0m
grape_leaf_DD:       • Leaf_blight: 33.02%␣[0m
grape_leaf_DD:       • Black_rot: 13.34%␣[0m
grape_leaf_DD:       • Esca: 11.24%␣[0m
```

Figure: ESP32-S3 serial output showing on-device inference and detection results per-leaf disease probability scores.

Next Steps

Since both models are successfully deployed onto the device, the upcoming phase will focus on:

- **LoRaWAN Network Integration**

- Connect ESP32-S3 nodes → STM32 Nucleo Node → LoRaWAN
- Implement efficient data transmission protocol
- Test coverage and reliability in vineyard

- **Power Analysis**

- Measure real-world power consumption
- Optimize sleep/wake cycles

- **Field Testing & Validation**

- Deploy multiple nodes across vineyard
- Validate aggregation accuracy in real conditions
- Collect long-term performance data

References



Xu Tao, Jackson Butcher, Simone Silvestri, and Sajal K. Das (2025)
iCrop+: An Edge-boosted Crop Disease Detection System via TinyML and LoRa Communication.

In 2025 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops), pp. 558–560.



Md Jawadul Karim, Md Omaer Faruq Goni, Md Nahiduzzaman, Mominul Ahsan, Julfikar Haider, and Marcin Kowalski (2024)

Enhancing agriculture through real-time grape leaf disease classification via an edge device with a lightweight CNN architecture and Grad-CAM.

Scientific Reports, 14(1), 16022.



Ning Zhang, Enxu Zhang, Guowei Qi, Fei Li, and Cheng Lv (2025)

Lightweight grape leaf disease recognition method based on transformer framework.

Scientific Reports, 15(1), 28974.



Grape Leaf Disease Dataset:

<https://www.kaggle.com/datasets/bmshahriaalam/grape-leaf-disease>



Thank You

Questions?